FINAL YEAR 2022/2023 FORMAL METHODS

SECTION 1

The case below shall formalize some of the ideas in the management of bank accounts. In the banking system, a customer's account is identified by an account number.

a) Write a declaration to introduce AcctNo as a given set for account numbers.

[AcctNo]

b) Write a single declaration to introduce the following sets:

active - is the set of account numbers that are in use. (the other account numbers might be yet to be allocated, or might be the numbers of accounts that have been closed).

overdrawn - is the set of account numbers of the accounts that are overdrawn

depositors - is the set of account numbers of deposit accounts, i.e. accounts that receive interest.

current - is the set of account numbers of current accounts, i.e. accounts that receive no interest.

$active, overdrawn, depositors, current : \mathbb{P}\, AcctNo$

c) Write a predicate to formalize the following statements:

i) only active accounts can be overdrawn.

$overdrawn \subseteq active$

ii) only current accounts can be overdrawn.

$overdrawn \subseteq current$

iii) the active accounts are the deposit accounts and the current accounts taken together.

$active = depositors \cup current$

iv) no account can be both a deposit account and a current account.

$Depositors \cap current = \emptyset$

d) Let's assume that the balance in each account is recorded by a partial function balance:AcctNo→Z, where integer is used to record sums of money.

Write a predicate to formalize the following statements:

i) the active accounts are those for which there are balances.

active = dom balance

ii) overdrawn accounts are those accounts that have negative balances

overdrawn = {a:AcctNo | balance a<0}

SECTION 2

CIMB_Response :: = no_more_account_numbers | overdrawn_balance_not_allowed | account_opened

CIMB_Bank

$active, depositors: \mathbb{P}\ AcctNo$
$active, depositors: \mathbb{P}\ AcctNo$
$balance: AcctNo \rightarrow \mathbb{Z}$

$current \cup depositors = active$
$current \cap depositors = \varnothing$
$active = \mathbf{dom}\ balance$
$overdrawn = \{ a: current \mid balance\ a < 0 \}$
$\forall a: depositors \bullet balance\ a \geq 0$

Table 1: Informal Specification of the CIMB bank account's management system

Inputs	Processing	Outputs
Open a deposit account		
amount	If there are no account numbers left, then the response is 'no more account numbers'. If the amount offered is negative, then the response is 'overdrawn balances not allowed'. Otherwise the account is opened in the input amount with an unused account number chosen by the system, and the response is 'account opened'. The account number chosen is displayed.	response, account number
Open a current account		
amount	If there are no account numbers left, then the response is 'no more account numbers'. Otherwise the account is opened in the input amount with an unused account number chosen by the system, and the response is 'account opened'. The account number chosen is displayed.	response, account number

a) Open a deposit account

----OpenDepositAccount-------

$\Delta CIMB_Bank$

$amount? : \mathbb{Z}$

$response! : CIMB_Response$

$accountNumber! : AcctNo$

$amount? \geq 0$ a

$accountNumber! \notin active$

$balance' = balance \oplus \{accountNumber! \mapsto amount?\}$

$active' = active \cup \{accountNumber!\}$

$depositors' = depositors \cup \{accountNumber!\}$

$current' = current$

$response! = account_opened$

---

b) Open a current account

-----OpenCurrentAccount-------

ΔCIMB_Bank

amount? : $\mathbb{Z}$

response! : CIMB_Response

accountNumber! : AcctNo

---

accountNumber! $\notin$ active

balance' = balance $\oplus$ {accountNumber! $\mapsto$ amount?}

active' = active $\cup$ {accountNumber!}

current' = current $\cup$ {accountNumber!}

depositors' = depositors

response! = account_opened

SECTION 3

[FYP_Report]

MESSAGE ::= *is_in_stock* | *is_booked_out*

- a set *all_reports* of objects of type *FYP_Report*
- a subset of them *in_stock* which are held at the centre
- a subset *booked_out* which have been booked out

A specification for the FIK FYP Report Centre has a state schema as below:

```
┌── FFR_Centre ──────────────────────
│ all_reports, in_stock, booked_out: ℙ FYP_Report
├────────────────────────
│ in_stock ∪ booked_out = all_reports
└────────────────────────────────────
```

a. Borrowing a FYP's Report: students must be able to make reservation for a required report.

----BorrowReport-------

ΔFFR_Centre

report? : FYP_Report

---

report? ∈ in_stock

in_stock' = in_stock \ {report?}

booked_out' = booked_out ∪ {report?}

all_report' = all_report

---

b. Returning a FYP'S Report: students must return the loan report.

-----ReturnReport------

ΔFFR_Centre

report? : FYP_Report

---

report? ∈ booked_out

booked_out' = booked_out \ {report?}

in_stock' = in_stock ∪ {report?}

all_report’ = all_report

---

c. Remove of a FYP'S Report: administrator must be able to remove an inappropriate report from the system.

-----RemoveReport-----

ΔFFR_Centre

report? : FYP_Report

---

report? ∈ all_reports

all_reports' = all_reports \ {report?}

in_stock' = in_stock \ {report?}

booked_out' = booked_out \ {report?}

---

d. Find FYP's Report: both students and administrator able to search and view the selected report.

----FindReport----------

ΞFFR_Centre

report? : FYP_Report

message! : MESSAGE

---

report? $\in$ all_reports

report? $\in$ in_stock $\Rightarrow$ message! = is_in_stock

report? $\in$ booked_out $\Rightarrow$ message! = is_booked_out

---

Sample 1 Gym Membership Management System

State Schema :

GymSystem

members: ℙ MEMBER

classes: ℙ CLASS

assigned: MEMBER ⇸ CLASS

completed: CLASS ⇸ MEMBER

SCHEMA 1: RegisterMember

RegisterMember

ΔGymSystem

m?: MEMBER

---

m? ∉ members

members' = members ∪ {m?}

classes' = classes

assigned' = assigned

completed' = completed

SCHEMA 2: AssignClass

AssignClass

ΔGymSystem

m?: MEMBER

c?: CLASS

---

m? ∈ members

c? ∈ classes

m? ∉ dom assigned

assigned' = assigned ∪ {m? ↦ c?}

members' = members

classes' = classes

completed' = completed

SCHEMA 3: MarkCompleted

MarkCompleted

ΔGymSystem

m?: MEMBER

c?: CLASS

---

m? ∈ members

c? ∈ classes

(m? ↦ c?) ∈ assigned

c? ∉ dom completed

completed' = completed ∪ {c? ↦ m?}

members' = members

classes' = classes

assigned' = assigned

Sample 2 : Sport Event

a) - Write a schema JoinGames which records a new university joining in the SUKUM games 2014

State Schema :

participating: $\mathbb{P}$ UNIVERSITY

event: $\mathbb{P}$ EVENT

represents: PERSON $\rightarrowtail$ UNIVERSITY

competes_in: PERSON $\leftrightarrow$ EVENT

won: EVENT $\rightarrow$ PERSON

a) Schema: JoinGames

JoinGames

ΔSUKUM2014

u?: UNIVERSITY

---

u? $\notin$ participating

participating' = participating $\cup$ {u?}

event' = event

represents' = represents

competes_in' = competes_in

won' = won

b)- Write a schema Win That records the winner of an event

b) Schema: Win

Win

ΔSUKUM2014

e?: EVENT

p?: PERSON

---

e? $\in$ event

e? $\notin$ dom won

won' = won $\cup$ {e? $\mapsto$ p?}

participating' = participating

event' = event

represents' = represents

competes_in' = competes_in

<u>Sample 3 : Library Book Borrowing</u>

State Schema

Library

borrowedBy: BOOK ⇸ STUDENT

a) BorrowBook

Record a borrowing made by a student for a book.

BorrowBook

ΔLibrary

b?: BOOK

s?: STUDENT

---

b? ∉ dom borrowedBy

borrowedBy' = borrowedBy ∪ {b? ↦ s?}

b) ReturnBook

Cancel a borrowing (return the book).

ReturnBook

ΔLibrary

b?: BOOK

s?: STUDENT

(b? ↦ s?) ∈ borrowedBy

borrowedBy' = borrowedBy \ {(b? ↦ s?)}

c) ChangeBorrower

Update the borrower of a book.

ChangeBorrower

ΔLibrary

b?: BOOK

s?: STUDENT

---

b? ∈ dom borrowedBy

borrowedBy' = borrowedBy ⊕ {b? ↦ s?}

<u>Sample 4 : Booking Classroom</u>

<u>Given State Schema</u>

Class

bookedTo: CLASSROOM ⇸ LECTURER

Practice a) **AcceptBooking**

a) Write schema AcceptBooking which records a booking made from a given person and a given classroom

AcceptBooking

ΔClass

c?: CLASSROOM

l?: LECTURER

---

c? ∉ dom bookedTo

bookedTo' = bookedTo ∪ {c? ↦ l?}

b) CancelBooking

B)- Write schema CancelBooking which cencel records a booking made from a given person and a given classroom

---

CancelBooking

ΔClass

c?: CLASSROOM

l?: LECTURER

(c? ↦ l?) ∈ bookedTo

bookedTo' = bookedTo \ {(c? ↦ l?)}

c) UpdateBooking

c) Write schema UpdateBooking which cencel records a booking made from a given person and a given classroom

---

UpdateBoking

ΔCLAS

c?: CLASSROOM

l?: LECTURER

---

c? ∈ dom bookedTo

bookedTo' = bookedTo ⊕ {c? ↦ l?}

Exercise

Let **[Person]** be the set of all possible persons and **[Subjects]** the set of all possible subjects. A specification for an *University* has a state schema:

```
┌── University ──────────────────
│ students   : ℙ Person
│ subject    : ℙ Subjects
│ enrolments : Person ⇸ Subjects
├────────────────────────────────
│ dom enrolments ⊆ students
│ ran enrolments ⊆ subjects
└────────────────────────────────
```

a) An operation schema *newStudent* such that a person becomes a student.

```
┌── newStudent ──────────────────
│ ΔUniversity
│ p?:Person
├────────────────────────────────
│ p? ∉ students
│ student' = students ∪ {p?}
└────────────────────────────────
```

b) An operation schema *studentLeaves* such that a person, not enrolled in any subject.

```
┌── studentLeaves ───────────────
│ ΔUniversity
│ p? : Person
├────────────────────────────────
│ p? ∈ students
│ p? ∉ dom enrolments
│ students' = students \ {p?}
└────────────────────────────────
```

c) An operation schema *cancelEnrolments* such that a person cancels enrolments in a subject.

```
┌── cancelEnrolments ────────────
│ ΔUniversity
│ p? : Person
│ s? : Subjects
├────────────────────────────────
│ s? ∈ subject
│ enrolments' = enrolments \ {(p? ↦ s?)}
└────────────────────────────────
```

d) An operation schema *personSubjects* which outputs the set of subjects in which the person is enrolled.

```
┌── personSubjects ──────────────
│ ΞUniversity
│ p? : Person
│ s! : ℙ Subjects
├────────────────────────────────
│ p? ∈ students
│ s! = ran{(p? ◁ enrolments )}
└────────────────────────────────
```

Final Exam 2023 Questions :

Solution :

A) Open a deposit Account :

Deposit_account

$\Delta CIMB_Bank$
$ammount? : \mathbb{Z}$
$a! : AccNo$
$r! : CIMB_Response$

$a! \notin active$
$ammount? \geqslant 0$
$active' = active \cup \{a!\}$
$depositors' = depositors \cup \{a!\}$
$balance' = balance \oplus \{a! \mapsto ammount?\}$
$current' = current$
$r' = account_oponed$

B) Open Current Account

CurrentAccount

$\Delta CIMB_Bank$
$ammount?:\mathbb{Z}$
$a!:AccNo$
$r!:CIMB_Response$

$a! \notin active$
$active' = active \cup \{a!\}$
$current' = current \cup \{a!\}$
$balance' = balance \oplus \{a! \mapsto ammount?\}$
$depositors'' = depositors$
$r' = accouint\ opened$

Final Exam 2023 Questions :

Solution :

a) Borrowing a FYP Report : students must be able to make reservation for a required report

$$
\begin{array}{l}
\textit{FYP_Report_booked_out} \\
\hline
\Delta FFR_Centre \\
rep?:FYP_Report \\
\hline
rep? \notin booked_out \\
rep? \in in_stock \\
booked_out = booked_out \cup \{rep?\} \\
in_stock' = in_stock \setminus \{rep?\} \\
all_report' = all_report \\
\hline
\end{array}
$$

b) Returning FYP report : students must return the loan report :

$$
\begin{array}{l}
\textit{FYP_Report_returned} \\
\hline
\Delta FFR_Centre \\
rep?:FYP_Report \\
\hline
rep? \in booked_out \\
booked_out' = booked_out \setminus \{rep?\} \\
in_stock' = in_stock \cup \{rep?\} \\
all_report' = all_report \\
\hline
\end{array}
$$

c) Remove of a FYP Report : administrator must be able to remove an inappropriate report from the system

$$
\begin{array}{l}
\textit{Remove_FYP_Report} \\
\hline
\Delta FFR_Centre \\
rep? : FYP_Report \\
\hline
rep? \in all_reports \\
all_reports' = all_reports \setminus \{rep?\} \\
booked_out' = booked_out \setminus \{rep?\} \\
in_stock' = in_stock \setminus \{rep?\} \\
\hline
\end{array}
$$

d) Find FYP report both students and admistrator to search and view the selected report

$$
\begin{array}{l}
\textit{Find_FYP_Report} \\
\hline
\Xi FFR_Centre \\
rep? : FYP_Report \\
message! : MESSAGE \\
\hline
rep? \in all_reports \\
rep? \in in_stock \Rightarrow message! = is_in_stock \\
rep? \notin in_stock \Rightarrow message! = is_booked_out \\
\hline
\end{array}
$$

e) List FYP Report : An Operation that represting listing of Videeos in the shop's stock

$$
\begin{array}{l}
\textit{List_Report_FYP} \\
\hline
\Xi\ FFR_Centre \\
list! : \mathbb{P}\ FYP_report \\
\hline
list! = all_reports \\
\hline
\end{array}
$$

Question 2 [25 Marks]

The *Lydia Transit Inn* is a preferable accommodation among tourists that traveling in Kuala Terengganu. It provides a simple room reservation system to record the guest accomodation data such as check in date and check out date. The data type of this system are introduced as in given set as follows:

[DATE]	the set of all reservation dates
[GUEST]	the set of all guests

A state schema for the *Lydia Transit Inn* in Z notation is as below:

```
┌─ LydiaT_Inn ──────────
│ bookedTo: DATE → GUEST
└───────────────────────
```

Construct the Z specification to formalize the following operations:

i. Room reservation [8 Marks]

ii. Cancel reservation [8 Marks]

iii. Update reservation [9 Marks]

a) Room Reservation

```
┌─ Room_Reservation ────────────
│ ΔLydiaT_Inn
│ d?:DATE
│ g?:GUEST
├───────────────────────────
│ d? ∉ dom bookedTo
│ bookedTo' = bookedTo ∪ { d? ↦ g?}
└───────────────────────────
```

b) Cancel Reservation

```
┌─ Cancel_Reservation ──────────
│ ΔLydiaT_Inn
│ d?: DATE
│ g?: GUEST
├───────────────────────────
│ {d? ↦ g} ∈ bookedTo
│ bookedTo' = bookedTo \ {d? ↦ g?}
└───────────────────────────
```

c) Update Reservation

```
┌─ Update_Reservation ──────────
│ ΔLydiaT_Inn
│ d?:DATE
│ g?GUEST
├───────────────────────────
│ d? ∈ dom bookedTo
│ bookedTo = bookedTo ⊕ { d?↦g?}
└───────────────────────────
```

Question 1 [20 Marks]

Write a Z schema to represent the following case study:-

a) A simple banking system with entities such as customers, accounts, transactions, and balance. Include the necessary sets, relations, and schema variables. [6 Marks]

b) A simple online shopping system with entities such as products, customers, orders, and orders status. Define the necessary sets, relations, and schema variables. [7 Marks]

c) A library management system with entities such as books, borrowers, loans, and loans status. Define the necessary sets, relations, and schema variables. [7 Marks]

Question 2 [25 Marks]

(a) Simple Banking System [6 Marks]
[CUSTOMER, ACCOUNT, TRANSACTION]

```
BankSystem
  customers: ℙ CUSTOMER
  accounts: ℙ ACCOUNT
  transactions: ℙ TRANSACTION
  owns: CUSTOMER ↔ ACCOUNT
  balance: ACCOUNT → ℤ
  transHistory: ACCOUNT → ℙ TRANSACTION

  -- Each account must belong to at most one customer
  ∀ a: ACCOUNT • #(owns~[{a}]) ≤ 1
```

Explanation:

- `CUSTOMER`, `ACCOUNT`, `TRANSACTION` are given sets.
- `owns`: relationship showing which customer owns which account.
- `balance`: maps each account to an integer representing money.
- `transHistory`: maps accounts to sets of transactions.
- Constraint: An account can be owned by at most one customer.

(b) Simple Online Shopping System [7 Marks]

```
[ CUSTOMER, PRODUCT, ORDER ]

OrderStatus ::= pending | shipped | delivered | cancelled

OnlineShop
   customers: ℙ CUSTOMER
   products: ℙ PRODUCT
   orders: ℙ ORDER
   placedBy: ORDER → CUSTOMER
   contains: ORDER → ℙ PRODUCT
   status: ORDER → OrderStatus

   -- Every order must be placed by a customer and contain at least one product
   ∀ o: ORDER • o ∈ orders ⇒ placedBy o ∈ customers ∧ contains o ≠ ∅
```

Explanation:

- Defines basic sets and an enumerated type `OrderStatus`.
- `placedBy`: maps each order to a customer.
- `contains`: shows which products are in an order.
- `status`: current status of each order.
- Constraint: all orders are linked to a customer and have products.

(c) Library Management System [7 Marks]

```
[ BOOK, BORROWER, LOAN ]

LoanStatus ::= active | returned | overdue

LibrarySystem
   books: ℙ BOOK
   borrowers: ℙ BORROWER
   loans: ℙ LOAN
   loanedBook: LOAN → BOOK
   loanedTo: LOAN → BORROWER
   loanStatus: LOAN → LoanStatus

   -- Each loan must be linked to a book and borrower
   ∀ l: LOAN • l ∈ loans ⇒ loanedBook l ∈ books ∧ loanedTo l ∈ borrowers
```

Explanation:

- Introduces an enumerated type `LoanStatus`.
- `loanedBook`: shows which book is loaned.
- `loanedTo`: maps a loan to the borrower.

- `loanStatus`: indicates status of each loan.
- Constraint: each loan is valid only if it has a book and borrower.